

Chapter 6 Model Building and Scoring for Prediction

In the preceding chapters, we have only scratched the surface of model building. Linear regression is a great initial approach to take to model building. In fact, in the realm of statistical models, linear regression (calculated by ordinary least squares) is the **best linear unbiased estimator**. The two key pieces to that previous statement are “best” and “unbiased.”

What does it mean to be **unbiased**? Each of the sample coefficients ($\hat{\beta}$'s) in the regression model are estimates of the true coefficients. Just like the statistics back in the EDA section, these sample coefficients have sampling distributions - specifically, normally distributed sampling distributions. The mean of the sampling distribution of $\hat{\beta}_j$ is the true (known) coefficient β_j . This means the coefficient is unbiased.

What does it mean to be **best**? *IF* the assumptions of ordinary least squares are met fully, then the sampling distributions of the coefficients in the model have the **minimum** variance of all unbiased estimators.

These two things combined seem like what we want in a model - estimating what we want (unbiased) and doing it in a way that has the minimum amount of variation (best among the unbiased). Again, these rely on the assumptions of linear regression holding true. Another approach to regression would be to use **regularized regression** instead as a different approach to building the model.

This Chapter aims to answer the following questions:

- What is regularized regression?
 - Penalties in Modeling
 - Ridge Regression
 - LASSO
 - Elastic Net
- How do you optimize the penalty term?
 - Overfitting
 - Cross-Validation (CV)
 - CV in Regularized Regression
- How do you compare different types of models?
 - Model Metric
 - Model Scoring
 - Test Dataset Comparison

6.1 Regularized Regression

As the number of variables in a linear regression model increase, the chances of having a model that meets all of the assumptions starts to diminish. Multicollinearity can pose a large problem with bigger regression models. As previously discussed, the coefficients of a linear regression vary widely in the presence of multicollinearity. These variations lead to overfitting of a regression model. **Overfitting** occurs when a dataset predicts the training data it was built off of really well, but does not generalize to the test dataset or the population in general. More formally, these models have higher variance than desired. In those scenarios, moving out of the realm of unbiased estimates may provide a lower variance in the model, even though the model is no longer unbiased as described above. We wouldn't want to be too biased, but some small degree of bias might improve the model's fit overall.

Another potential problem for linear regression is when we have more variables than observations in our dataset. This is a common problem in the space of genetic modeling. In this scenario, the ordinary least squares approach leads to multiple solutions instead of just one. Unfortunately, most of these infinite solutions overfit the problem at hand anyway.

Regularized (or penalized or shrinkage) regression techniques potentially alleviate these problems. Regularized regression puts constraints on the estimated coefficients in our model and *shrink* these estimates to zero. This helps reduce the variation in the coefficients (improving the variance of the model), but at the cost of biasing the coefficients. The specific constraints that are put on the regression inform the three common approaches - **ridge regression**, **LASSO**, and **elastic nets**.

6.1.1 Penalties in Models

In ordinary least squares linear regression, we minimize the sum of the squared residuals (or errors) .

$$\min\left(\sum_{i=1}^n (y_i - \hat{y}_i)^2\right) = \min(SSE)$$

In regularized regression, however, we add a penalty term to the SSE as follows:

$$\min\left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \text{Penalty}\right) = \min(SSE + \text{Penalty})$$

As mentioned above, the penalties we choose constrain the estimated coefficients in our model and shrink these estimates to zero. Different penalties have different effects on the estimated coefficients. Two common approaches to adding penalties are the ridge and LASSO approaches. The elastic net approach is a combination of these two. Let's explore each of these in further detail!

6.1.2 Ridge Regression

Ridge regression adds what is commonly referred to as an “ L_2 ” penalty:

$$\min\left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \hat{\beta}_j^2\right) = \min(SSE + \lambda \sum_{j=1}^p \hat{\beta}_j^2)$$

This penalty is controlled by the **tuning parameter** λ . If $\lambda = 0$, then we have typical OLS linear regression. However, as $\lambda \rightarrow \infty$, the coefficients in the model shrink to zero. This makes intuitive sense. Since the estimated coefficients, $\hat{\beta}_j$'s, are the only thing changing to minimize this equation, then as $\lambda \rightarrow \infty$, the equation is best minimized by forcing the coefficients to be smaller and smaller. We will see how to optimize this penalty term in a later section.

Let's build a regularized regression for our Ames dataset. To build a ridge regression we need separate data matrices for our predictors and our target variable. First, we isolate out the variables we are interested in using the `select` function. These functions cannot handle missing values in our dataset, so the last line of code using the `mutate_if` function which replaces missing values with the average of variable it is in. From there the `model.matrix` function will create any categorical dummy variables needed. We also isolate the target variable into its own vector.

```
train_reg <- train %>%  
  dplyr::select(Sale_Price,  
    Lot_Area,  
    Street,  
    Bldg_Type,  
    House_Style,  
    Overall_Qual,  
    Roof_Style,  
    Central_Air,  
    First_Flr_SF,  
    Second_Flr_SF,  
    Full_Bath,  
    Half_Bath,  
    Fireplaces,  
    Garage_Area,  
    Gr_Liv_Area,  
    TotRms_AbvGrd) %>%  
  mutate_if(is.numeric, ~replace_na(.,mean(., na.rm = TRUE)))  
  
train_x <- model.matrix(Sale_Price ~ ., data = train_reg)[, -1]  
train_y <- train_reg$Sale_Price
```

We will want to do the same thing for the test dataset as well.

```
test_reg <- test %>%
  dplyr::select(Sale_Price,
    Lot_Area,
    Street,
    Bldg_Type,
    House_Style,
    Overall_Qual,
    Roof_Style,
    Central_Air,
    First_Flr_SF,
    Second_Flr_SF,
    Full_Bath,
    Half_Bath,
    Fireplaces,
    Garage_Area,
    Gr_Liv_Area,
    TotRms_AbvGrd) %>%
  mutate_if(is.numeric, ~replace_na(., mean(., na.rm = TRUE)))

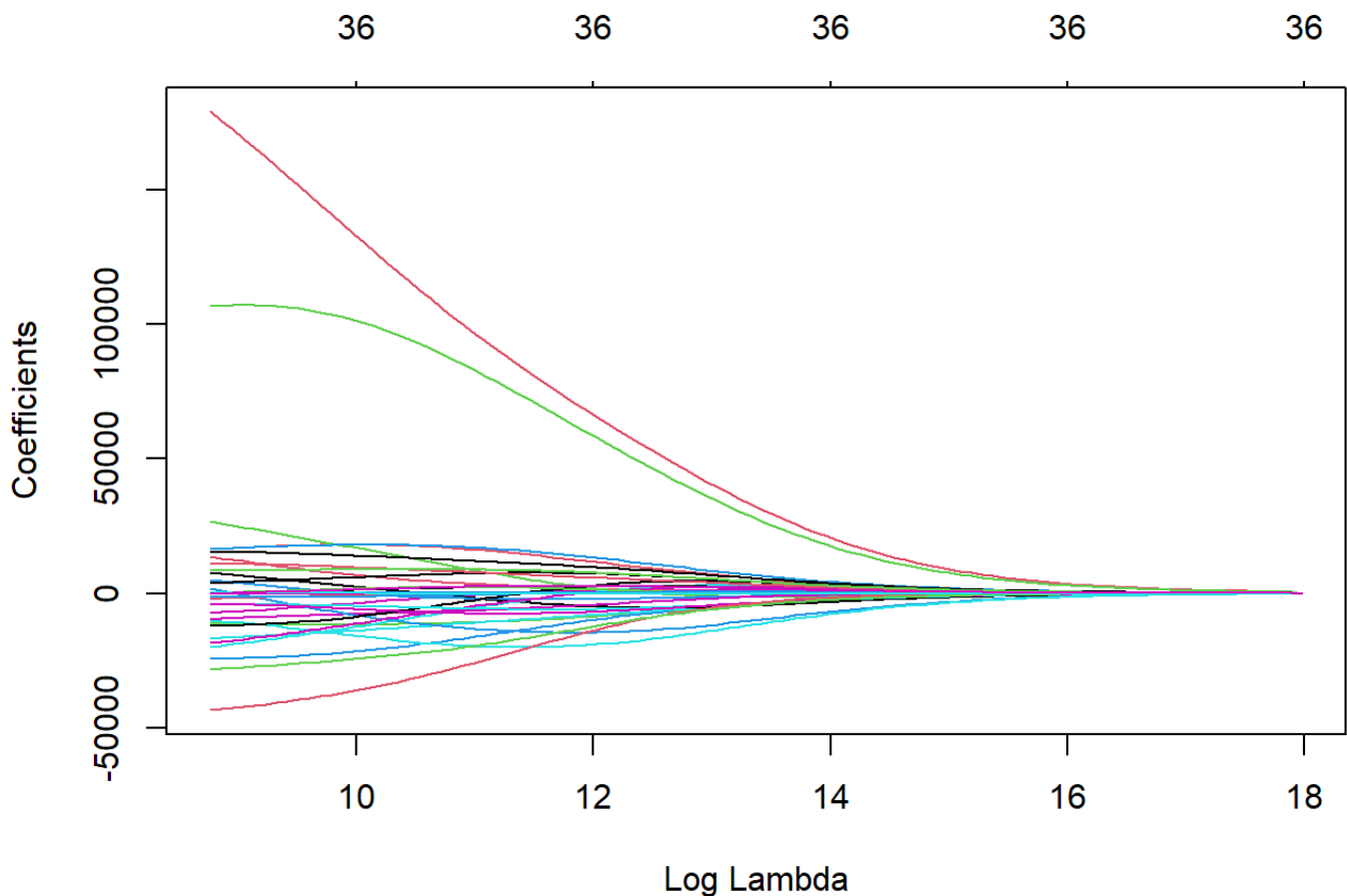
test_x <- model.matrix(Sale_Price ~ ., data = test_reg)[, -1]
test_y <- test_reg$Sale_Price
```

From there we use the `glmnet` function with the `x =` option where we specify the predictor model matrix and the `y =` option where we specify the target variable. The `alpha = 0` option specifies that a ridge regression will be used as defined in more detail below in the elastic net section. The `plot` function allows us to see the impact of the penalty on the coefficients in the model.

```
library(glmnet)

ames_ride <- glmnet(x = train_x, y = train_y, alpha = 0)

plot(ames_ride, xvar = "lambda")
```



The `glmnet` function automatically standardizes the variables before fitting the regression model. This is important so that all of the variables are on the same scale before adjustments are made to the estimated coefficients. Even with this standardization we can see the large coefficient values for some of the variables. The top of the plot lists how many variables are in the model at each value of penalty. This will never change for ridge regression, but does for LASSO.

What λ term is best? That will be discussed in the optimizing section below, but let's discuss other possible penalties first.

6.1.3 LASSO

Least absolute shrinkage and selection operator (LASSO) regression adds what is commonly referred to as an “ L_1 ” penalty:

$$\min\left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\hat{\beta}_j|\right) = \min(SSE + \lambda \sum_{j=1}^p |\hat{\beta}_j|)$$

This penalty is controlled by the **tuning parameter** λ . If $\lambda = 0$, then we have typical OLS linear regression. However, as $\lambda \rightarrow \infty$, the coefficients in the model shrink to zero. This makes intuitive sense. Since the estimated coefficients, $\hat{\beta}_j$'s, are the only thing changing to minimize this equation, then as $\lambda \rightarrow \infty$, the equation is best minimized by forcing the coefficients to be smaller and smaller. We will see how to optimize this penalty term in a later section.

However, unlike ridge regression that has the coefficient estimates approach zero asymptotically, in LASSO regression the coefficients can actually equal zero. This may not be as intuitive when initially looking at the penalty terms themselves. It becomes easier to see when dealing with the solutions to the coefficient estimates. Without going into too much mathematical detail, this is done by taking the derivative of the minimization function (objective function) and setting it equal to zero. From there we can determine the optimal solution for the estimated coefficients. In OLS regression the estimates for the coefficients can be shown to equal the following (in matrix form):

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

This changes in the presence of penalty terms. For ridge regression, the solution becomes the following:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T Y$$

There is no value for λ that can force the coefficients to be zero by itself. Therefore, unless the data makes the coefficient zero, the penalty term can only force the estimated coefficient to zero asymptotically as $\lambda \rightarrow \infty$.

However, for LASSO, the solution becomes the following:

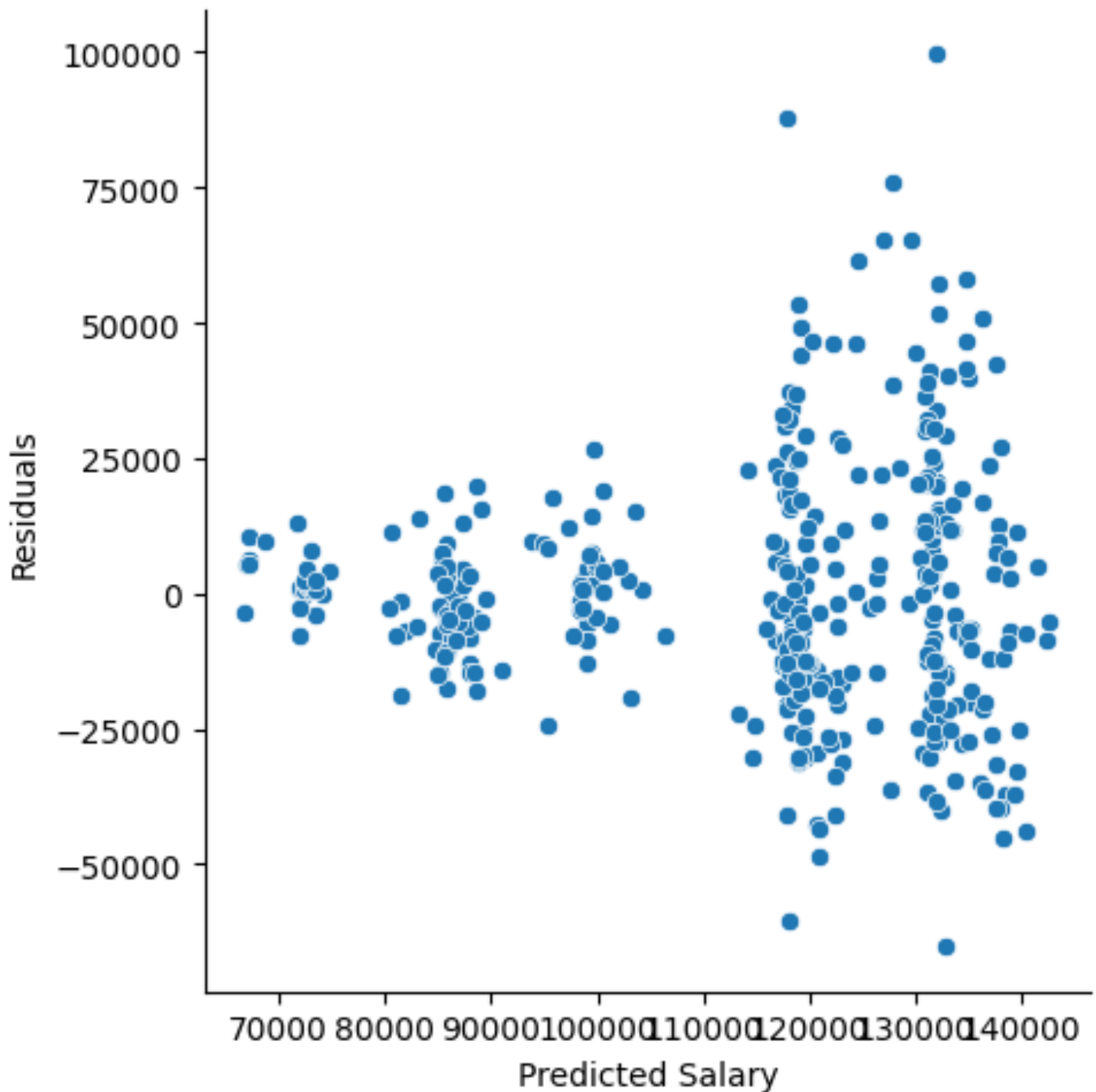
$$\hat{\beta} = (X^T X)^{-1}(X^T Y - \lambda I)$$

Notice the distinct difference here. In this scenario, there is a possible penalty value ($\lambda = X^T Y$) that will force the estimated coefficients to equal zero. There is some benefit to this. This makes LASSO also function as a variable selection criteria as well.

Let's build a regularized regression for our Ames dataset using the LASSO approach. To build a LASSO regression we need separate data matrices for our predictors and our target variable just like we did for ridge. From there we use the `glmnet` function with the `x =` option where we specify the predictor model matrix and the `y =` option where we specify the target variable. The `alpha = 1` option specifies that a LASSO regression will be used as defined in more detail below in the elastic net section. The `plot` function allows us to see the impact of the penalty on the coefficients in the model.

```
ames_lasso <- glmnet(x = train_x, y = train_y, alpha = 1)
```

```
plot(ames_lasso, xvar = "lambda")
```



The `glmnet` function automatically standardizes the variables before fitting the regression model. This is important so that all of the variables are on the same scale before adjustments are made to the estimated coefficients. Even with this standardization we can see the large coefficient values for some of the variables. The top of the plot lists how many variables are in the model at each value of penalty. Notice as the penalty increases, the number of variables decreases as variables are forced to zero.

What λ term is best? That will be discussed in the optimizing section below, but let's discuss the last possible penalty first - the combination of both ridge and LASSO.

6.1.4 Elastic Net

Which approach is better, ridge or LASSO? Both have advantages and disadvantages. LASSO performs variable selection while ridge keeps all variables in the model. However, reducing the number of variables might impact minimum error. Also, if you have two correlated variables, which one LASSO chooses to zero out is relatively arbitrary to the context of the problem.

Elastic nets were designed to take advantage of both penalty approaches. In elastic nets, we are using both penalties in the minimization:

$$\min(SSE + \lambda_1 \sum_{j=1}^p |\hat{\beta}_j| + \lambda_2 \sum_{j=1}^p \hat{\beta}_j^2)$$

In R, the `glmnet` function takes a slightly different approach to the elastic net implementation with the following:

$$\min(SSE + \lambda[\alpha \sum_{j=1}^p |\hat{\beta}_j| + (1 - \alpha) \sum_{j=1}^p \hat{\beta}_j^2])$$

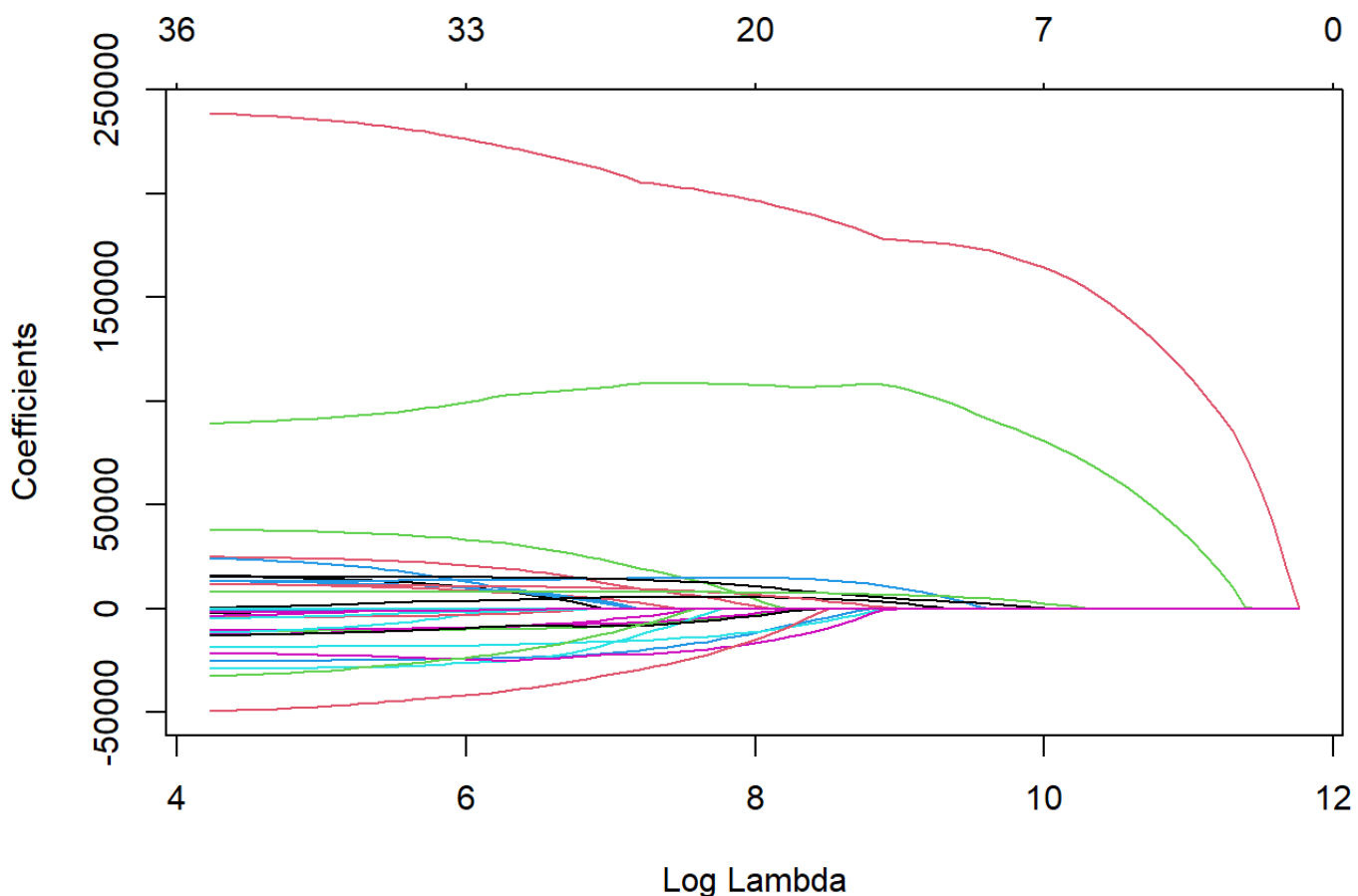
R still has one penalty λ , however, it includes the α parameter to balance between the two penalty terms. This is why in `glmnet`, the `alpha = 1` option gives a LASSO regression and `alpha = 0` gives a ridge regression. Any value in between zero and one will provide an elastic net.

Let's build a regularized regression for our Ames dataset using the elastic net approach with an $\alpha = 0.5$. To build an elastic net we need separate data matrices for our predictors and our target variable just like we did for ridge and LASSO. From there we use the `glmnet` function with the `x =` option where we specify the predictor model matrix and the `y =` option

where we specify the target variable. The `alpha = 0.5` option specifies that an elastic net will be used since it is between zero and one. The `plot` function allows us to see the impact of the penalty on the coefficients in the model.

```
ames_en <- glmnet(x = train_x, y = train_y, alpha = 0.5)
```

```
plot(ames_en, xvar = "lambda")
```



The `glmnet` function automatically standardizes the variables before fitting the regression model. This is important so that all of the variables are on the same scale before adjustments are made to the estimated coefficients. Even with this standardization we can see the large

coefficient values for some of the variables. The top of the plot lists how many variables are in the model at each value of penalty. Notice as the penalty increases, the number of variables decreases as variables are forced to zero using the LASSO piece of the elastic net penalty.

What λ term is best? What is the proper balance between ridge and LASSO penalties when building an elastic net? That will be discussed in the following section.

6.1.5 Python Code

Ridge Regression

```
import scipy as sp
import pandas as pd

## C:\Users\sjsimmo2\AppData\Local\Microsoft\Windows\Library\4.3\reticulate\python\rpytools\
##     return _find_and_load(name, import_)
## C:\Users\sjsimmo2\AppData\Local\Microsoft\Windows\Library\4.3\reticulate\python\rpytools\
##     return _find_and_load(name, import_)
```

```
import numpy as np
import statsmodels.formula.api as smf
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet

train=r.train
test=r.test
train = train[['Sale_Price', 'Lot_Area', 'Street', 'Bldg_Type',
               'House_Style', 'Overall_Qual', 'Roof_Style',
               'Central_Air', 'First_Flr_SF', 'Second_Flr_SF',
               'Full_Bath', 'Half_Bath', 'Fireplaces',
               'Garage_Area', 'Gr_Liv_Area', 'TotRms_AbvGrd']]

#train = train.fillna(0,inplace=True)

test = test[['Sale_Price', 'Lot_Area', 'Street', 'Bldg_Type',
              'House_Style', 'Overall_Qual', 'Roof_Style',
              'Central_Air', 'First_Flr_SF', 'Second_Flr_SF',
              'Full_Bath', 'Half_Bath', 'Fireplaces',
              'Garage_Area', 'Gr_Liv_Area', 'TotRms_AbvGrd']]

#test = test.fillna(0,inplace=True)
```

```
train_dummy = pd.get_dummies(train, columns = ['Street', 'Bldg_Type', 'House_Style'])

print(train_dummy)
```

```
##          Sale_Price  Lot_Area  ...  Central_Air_N  Central_Air_Y
## 0          232600      11216  ...           False           True
## 1          166000       2998  ...           False           True
## 2          170000      17871  ...           False           True
## 3          252000      10574  ...           False           True
## 4          134000       6000  ...           False           True
## ...           ...         ...  ...           ...           ...
## 2046         72500       6627  ...            True          False
## 2047        137000      11425  ...           False           True
## 2048         80000       5220  ...           False           True
## 2049        221000      12803  ...           False           True
## 2050        102000       7288  ...            True          False
##
## [2051 rows x 43 columns]
```

```
y = train_dummy['Sale_Price']
```

```
X = train_dummy.loc[:, train_dummy.columns != 'Sale_Price']
```

```
ames_ridge = Ridge(alpha = 1.0).fit(X, y)
```

```

n_alphas = 200
alphas = np.logspace(0.1, 5, n_alphas)

coefs = []
for a in alphas:
    ridge = Ridge(alpha=a)
    ridge.fit(X, y)
    coefs.append(ridge.coef_)

```

▼ Ridge

Ridge(alpha=100000.0)

```

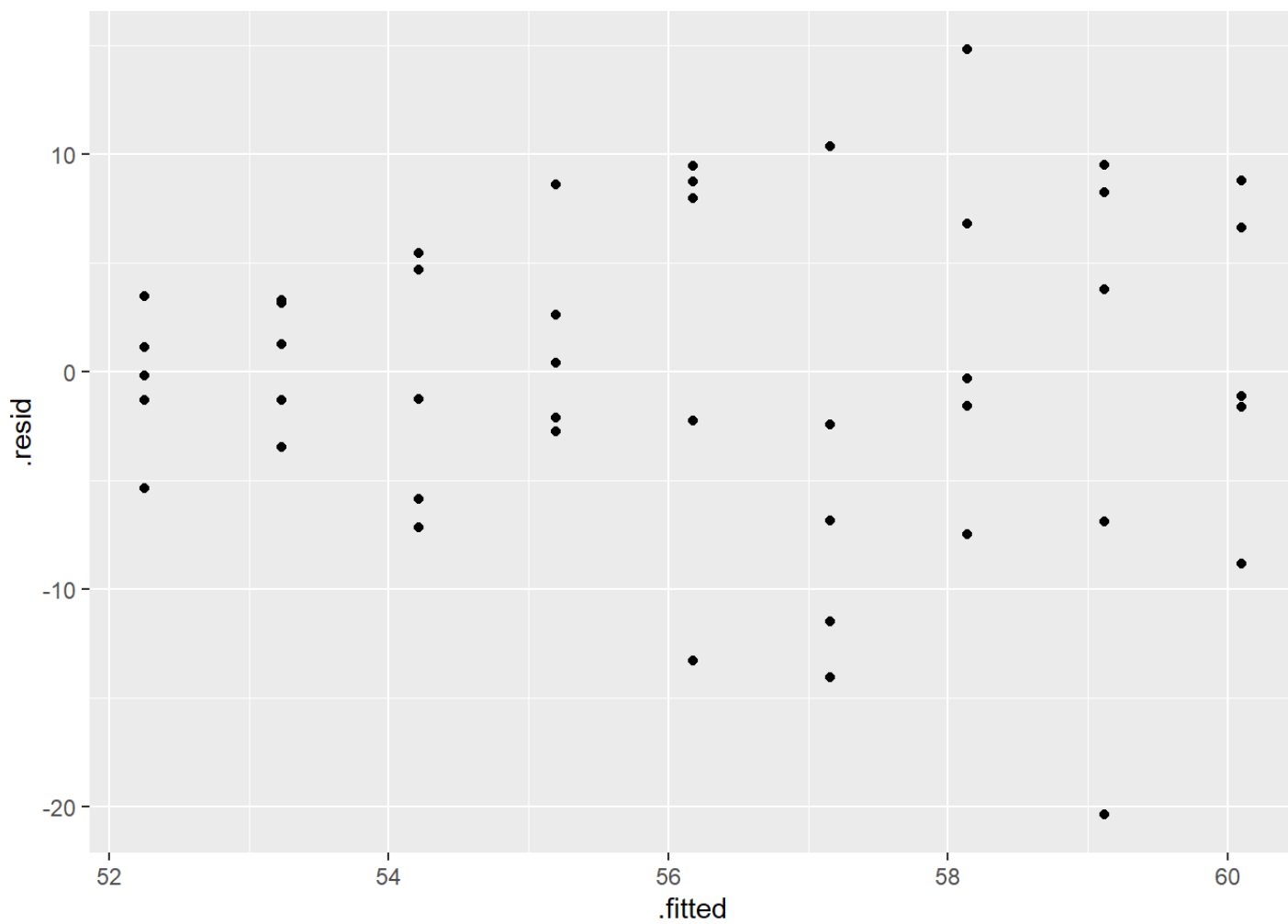
ax = plt.gca()

ax.plot(alphas, coefs)
ax.set_xscale("log")
plt.xlabel("alpha")
plt.ylabel("weights")
plt.title("Ridge coefficients as a function of the regularization")
plt.axis("tight")

## (0.716143410212902, 175792.36139586932, -75106.43079562689, 162751.9086572884)

plt.show()

```



LASSO Regression

```
ames_lasso = Lasso(alpha = 2, max_iter = 5000).fit(X, y)
```

```

n_alphas = 100
alphas2 = np.logspace(2, 5, n_alphas)

coefs2 = []
for a in alphas2:
    lasso = Lasso(alpha = a, max_iter = 5000)
    lasso.fit(X, y)
    coefs2.append(lasso.coef_)

```

▼ Lasso

```
Lasso(alpha=100000.0, max_iter=5000)
```

```

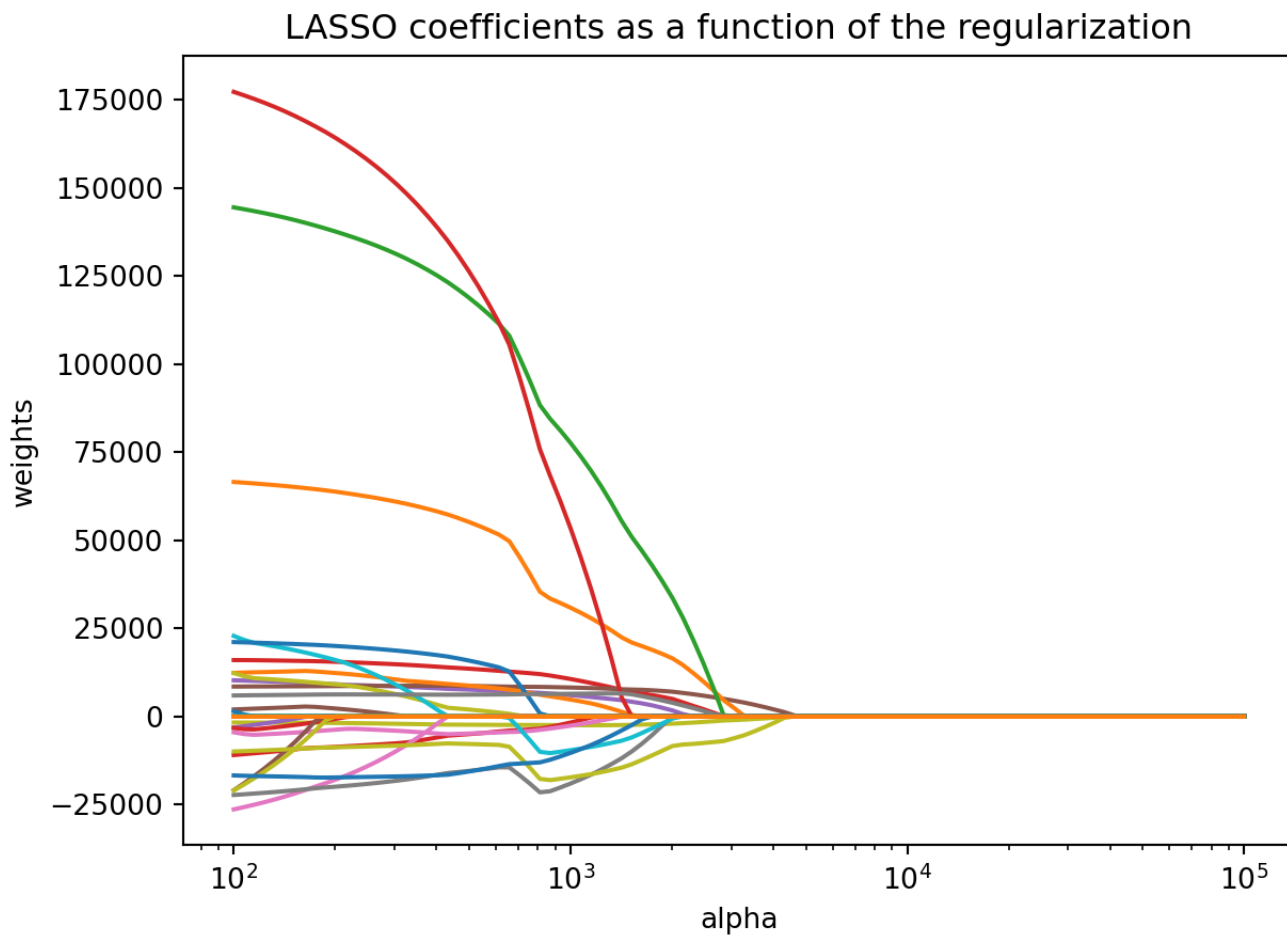
plt.cla()
ax2 = plt.gca()

ax2.plot(alphas2, coefs2)
ax2.set_xscale("log")
plt.xlabel("alpha")
plt.ylabel("weights")
plt.title("LASSO coefficients as a function of the regularization")
plt.axis("tight")

## (70.79457843841381, 141253.75446227554, -36612.70996531361, 187518.3833436268)

plt.show()

```



- Elastic Net*

```
ames_en = ElasticNet(alpha = 1, l1_ratio=0.5)
```

```

n_alphas = 100
alphas3 = np.logspace(1, 8, n_alphas)

coefs3 = []
for a in alphas3:
    en = ElasticNet(alpha = a, l1_ratio=0.5)
    en.fit(X, y)
    coefs3.append(en.coef_)

```

▼ ElasticNet

ElasticNet(alpha=100000000.0)

```

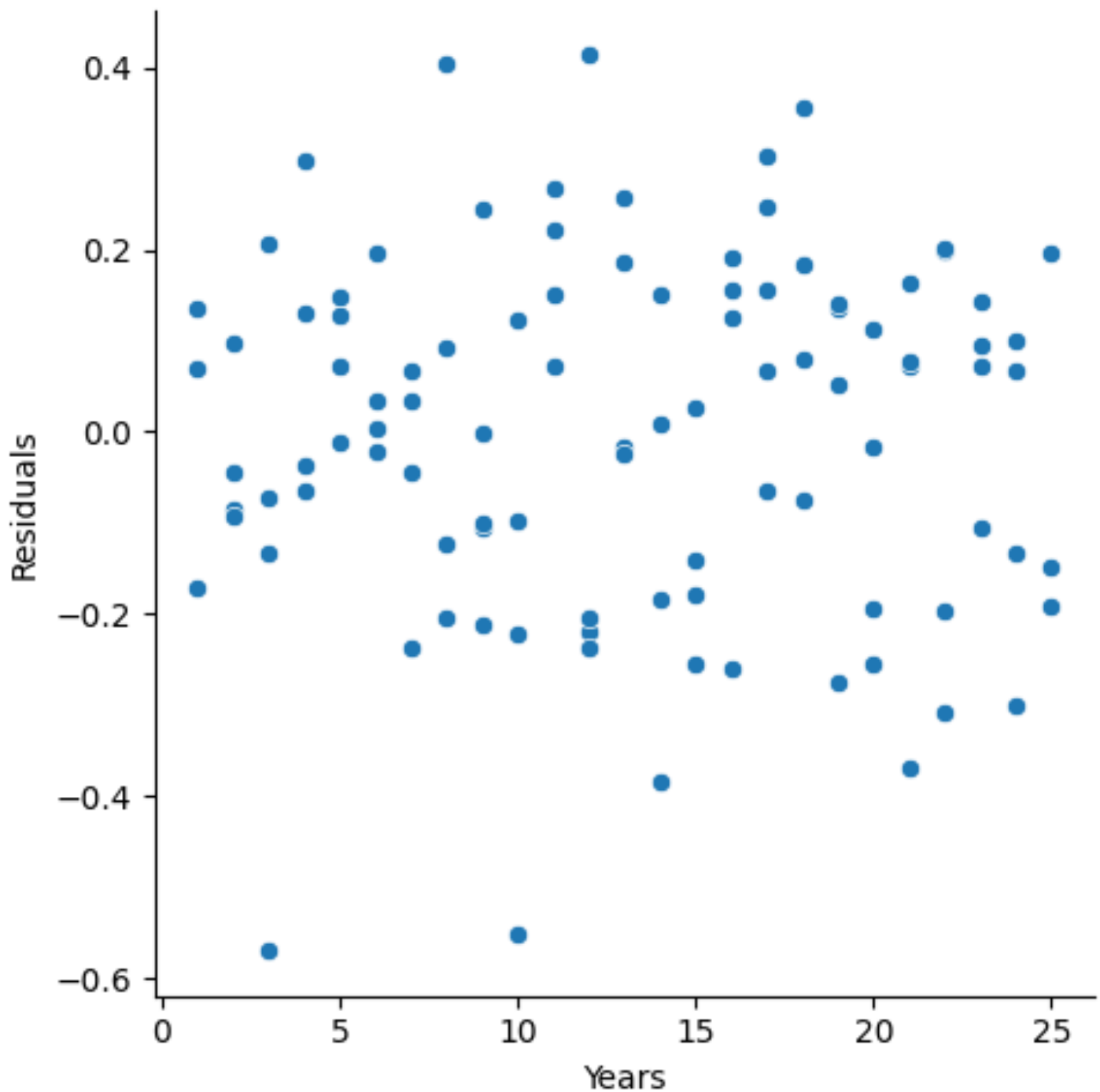
plt.cla()
ax3 = plt.gca()

ax3.plot(alphas3, coefs3)
ax3.set_xscale("log")
plt.xlabel("alpha")
plt.ylabel("weights")
plt.title("ElasticNet coefficients as a function of the regularization")
plt.axis("tight")

## (4.46683592150963, 223872113.8568338, -922.4074138311265, 951.0765209120292)

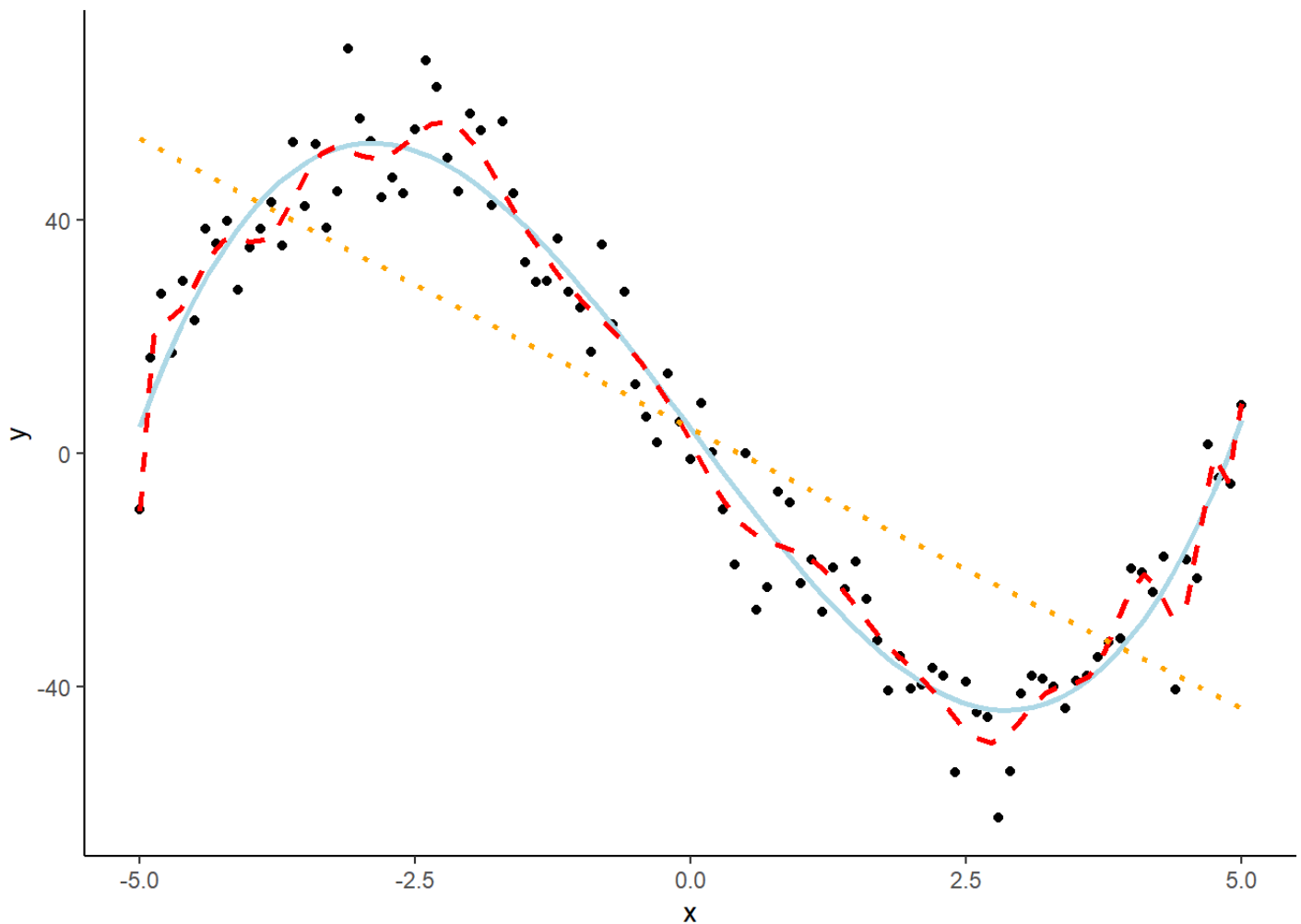
plt.show()

```



6.2 Optimizing Penalties

No matter the approach listed above, a penalty term λ needs to be picked. However, we do not want to get caught overfitting our training data by minimizing the variance so much that it is not generalizable to the overall pattern and other similar data. Take the following plot:



The red line is overfitted to the dataset and picks up too much of the unimportant pattern. The orange dotted line is underfit as it does not pick up enough of the pattern. The light blue, solid line is fit well to the dataset as it picks up the general pattern while not overfitting to the dataset.

6.2.1 Cross-Validation

Cross-validation is a common approach in modeling to prevent overfitting of data when you need to **tune** a parameter. The idea of cross-validation is to split the training data into multiple pieces, build the model on a majority of the pieces while evaluating it on the remaining piece. Then we do the same process again, but switch out which pieces the model is built and evaluated on.

A common cross-validation (CV) approach is the k -fold CV. In the k -fold CV approach, the model is built k times. The data is initially split into k equally sized pieces. Each time the model is built, it is built off of $k - 1$ pieces of the data and evaluated on the last piece. This process is repeated until each piece is left out for evaluation. This is diagrammed below in Figure 6.1.

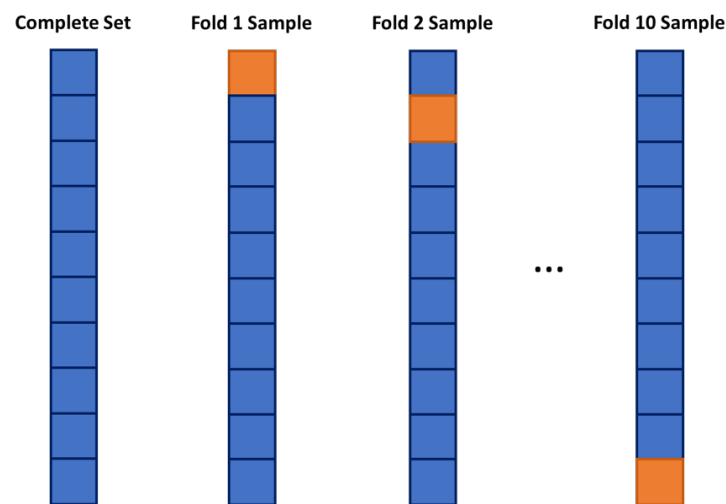


Figure 6.1: Example of a 10-fold Cross-Validation

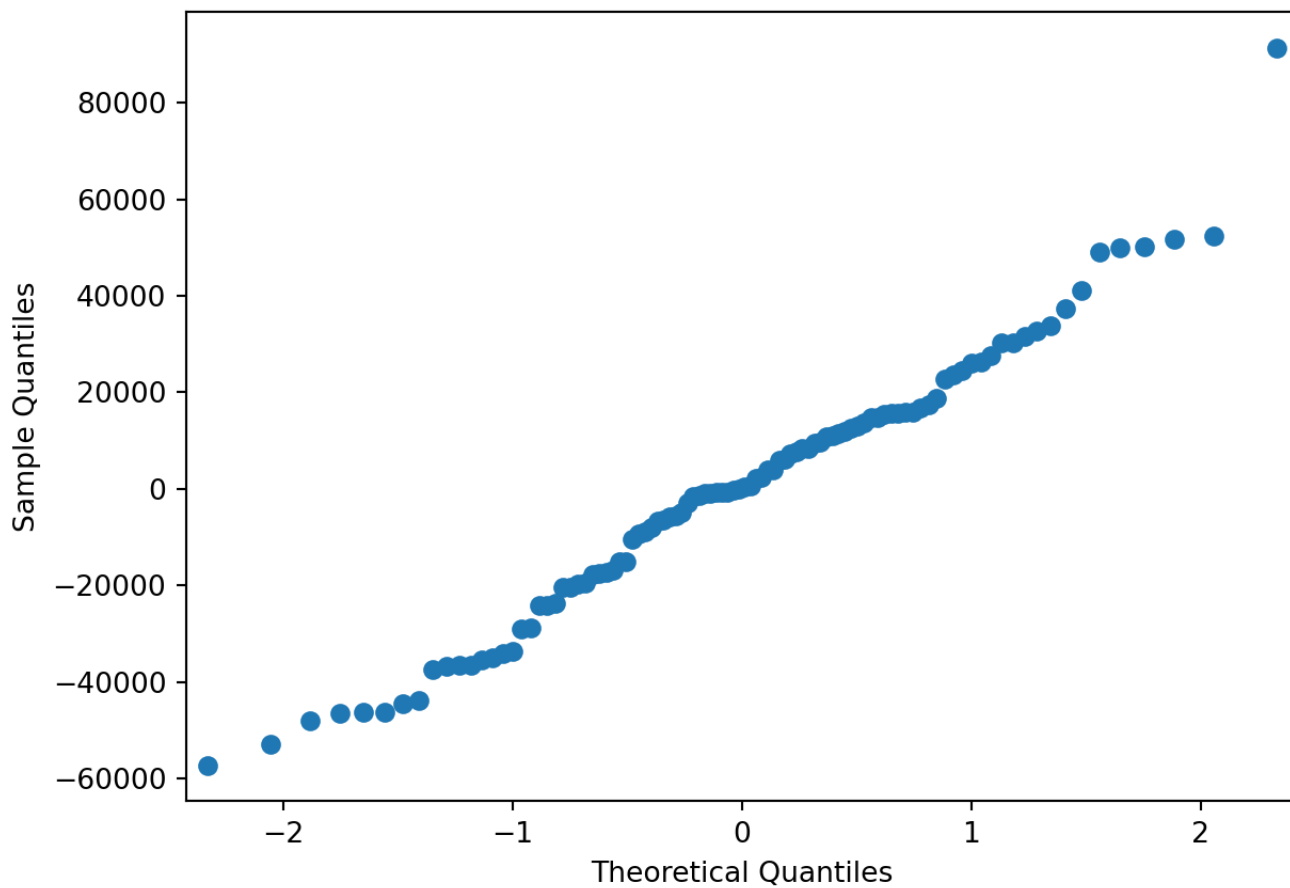
6.2.2 CV in Regularized Regression

In R, the `cv.glmnet` function will automatically implement a 10-fold CV (by default, but can be adjusted through options) to help evaluate and optimize the λ values for our regularized regression models.

Let's perform an example using the LASSO regression. The `cv.glmnet` function takes the same inputs as the `glmnet` function above. Again, we will use the `plot` function, but this time we get a different plot.

```
ames_lasso_cv <- cv.glmnet(x = train_x, y = train_y, alpha = 1)
```

```
plot(ames_lasso_cv)
```



```
ames_lasso_cv$lambda.min
```

```
## [1] 41.25712
```

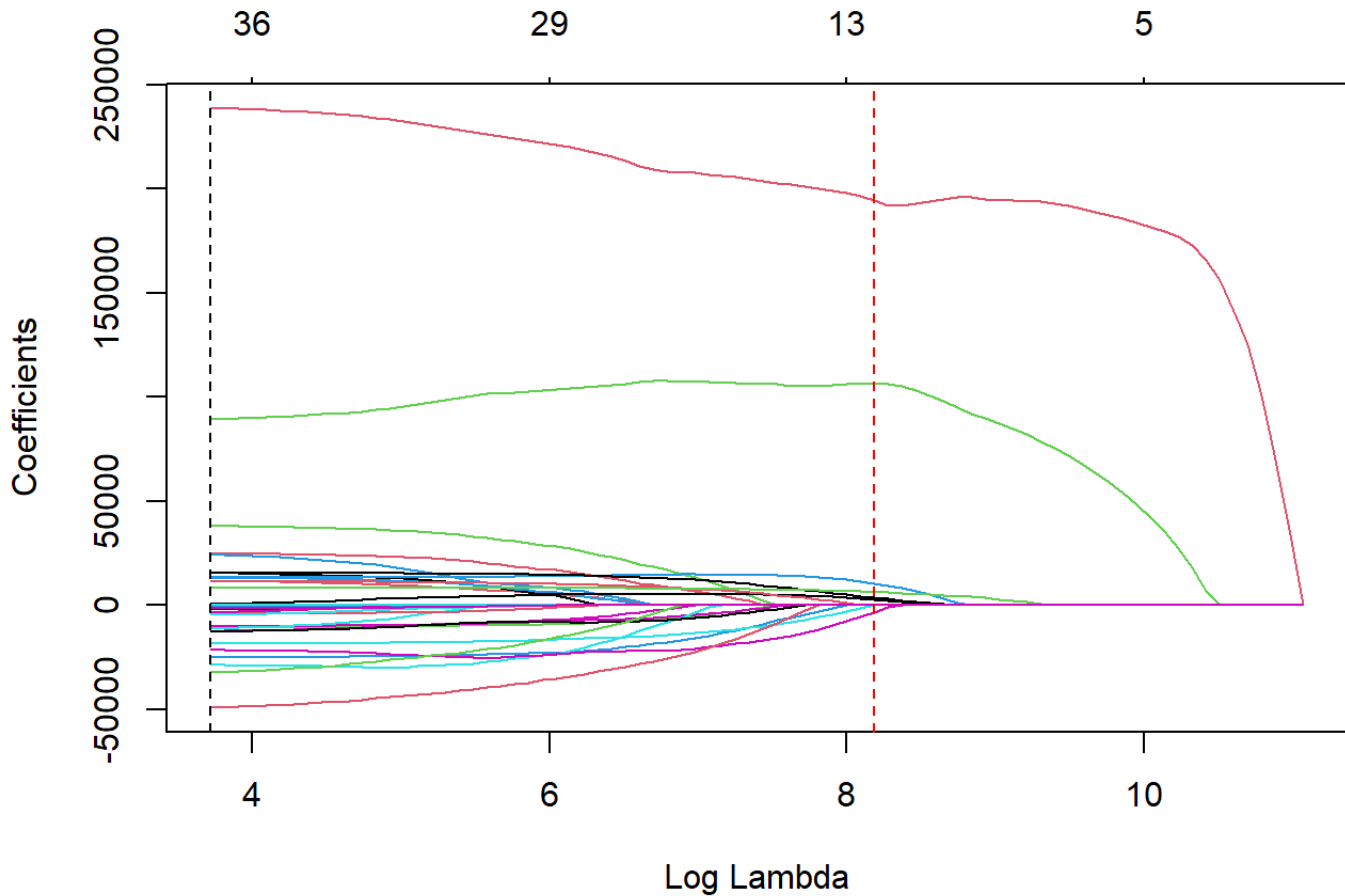
```
ames_lasso_cv$lambda.1se
```

```
## [1] 3588.334
```

The above plot shows the results from our cross-validation. Here the models are evaluated based on their **mean-squared error** (MSE). The MSE is defined as $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$. The λ value that minimizes the MSE is 49.69435 (with a $\log(\lambda) = 3.91$). This is highlighted by the first, vertical dashed line. The second vertical dashed line is the largest λ value that is one standard error above the minimum value. This value is especially useful in LASSO regressions. The largest λ within one standard error would provide approximately the same MSE, but with a further reduction in the number of variables. Notice that to go from the first line to the second, the change in MSE is very small, but the reduction of variables is from 36 variables to around 12 variables.

Let's look at the impact on the coefficients under this penalty using the `glmnet` function as before.

```
plot(ames_lasso, xvar = "lambda")
abline(v = log(ames_lasso_cv$lambda.1se), col = "red", lty = "dashed")
abline(v = log(ames_lasso_cv$lambda.min), col = "black", lty = "dashed")
```



To investigate which variables are important at a λ value, we can view the coefficients using the `coef` function. They are ranked here:

```
coef(ames_lasso, s = c(ames_lasso_cv$lambda.min, ames_lasso_cv$lambda.1se))
```

```
## 37 x 2 sparse Matrix of class "dgCMatrix"
```

##	s1	s2
## (Intercept)	4.553410e+04	84886.87520
## Lot_Area	5.549969e-01	0.34825
## StreetPave	1.169114e+04	.
## Bldg_TypeTwoFmCon	-1.071062e+04	.
## Bldg_TypeDuplex	-2.519976e+04	.

## Bldg_TypeTwnhs	-1.834311e+04	-32.78428
## Bldg_TypeTwnhsE	-1.017273e+04	.
## House_StyleOne_and_Half_Unf	1.547768e+04	.
## House_StyleOne_Story	2.494559e+04	.
## House_StyleSFoyer	3.808139e+04	.
## House_StyleSLvl	1.319978e+04	.
## House_StyleTwo_and_Half_Fin	-2.882582e+04	.
## House_StyleTwo_and_Half_Unf	-1.247721e+04	.
## House_StyleTwo_Story	-1.927207e+03	.
## Overall_Qual.L	2.387742e+05	194210.25872
## Overall_Qual.Q	8.927822e+04	106215.77903
## Overall_Qual.C	2.441868e+04	.
## Overall_Qual^4	-1.131683e+04	.
## Overall_Qual^5	-2.148416e+04	-3363.19922
## Overall_Qual^6	-1.270417e+04	.
## Overall_Qual^7	-4.028089e+03	.
## Overall_Qual^8	-2.797797e+02	.
## Overall_Qual^9	-8.008940e+02	.
## Roof_StyleGable	-4.698696e+03	.
## Roof_StyleGambrel	-3.415526e+03	.
## Roof_StyleHip	6.003091e+02	2802.76818
## Roof_StyleMansard	-4.917768e+04	.
## Roof_StyleShed	-3.222639e+04	.
## Central_AirY	1.324307e+04	10542.22209
## First_Flr_SF	1.641373e+01	14.63109
## Second_Flr_SF	2.756656e+01	.
## Full_Bath	1.558308e+04	3700.51328
## Half_Bath	1.153750e+04	.
## Fireplaces	8.079835e+03	6361.93458
## Garage_Area	3.523952e+01	43.01506
## Gr_Liv_Area	2.698449e+01	33.04945

```
## TotRms_AbvGrd          -1.486941e+03      .
```

The variable describing the overall quality of the home is the driving factor of this model as well as the other variables listed above.

A similar approach can be taken for CV with ridge regression using the same structure of code. That will not be covered here. Elastic nets are more complicated in that they have multiple parameters to optimize. For that approach, an optimization grid will need to be structured to evaluate different λ values across different α values. A loop can be set up to run the `cv.glmnet` function across many different values of α . That will not be covered in detail here.

6.3 Model Comparisons

Now we have multiple models built for our dataset. To help evaluate which model is better, we will use the test dataset as described in Chapter 2.

The models we have built are nothing but formulas. All we have to do is put the test dataset in the formula to predict/score the test data. We **do not** rerun the algorithm as the goal is **not** to fit the test dataset, but to just score it. We need to make sure that we have the same structure to the test dataset that we do with the training dataset. Any variable transformations, new variable creations, and missing value imputations done on the training dataset must be done on the test dataset in the same way.

6.3.1 Model Metrics

Once the predicted values are obtained from each model we need to evaluate good these predictions are. There are many different metrics to evaluate models depending on what type of target variable that you have. Some common metrics for continuous target variables are the square root of the mean squared error (RMSE), the mean absolute error (MAE), and mean absolute percentage error (MAPE).

The RMSE is evaluated as follows:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

The RMSE is an approximation of the standard deviation of the prediction errors of the model. The downside of the RMSE is a lack of interpretability.

The MAE is evaluated as follows:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

The MAE gives the average absolute difference between our predictions and the actual values. This is a symmetric measure with great interpretability. The main disadvantage of this metric is that it depends on the scale of the data. For comparing two models evaluated on the same data, this isn't important. However, when comparing across different datasets, this may not be as helpful. For example, in temperature predictions, having an MAE of five degrees for a model built on Honolulu, Hawaii weather might not be comparable to a model built on weather in Raleigh, North Carolina.

The MAPE is evaluated as follows:

$$MAPE = 100 \times \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

The MAPE gives the average absolute *percentage* difference between our predictions and the actual values. This metric is very interpretable and not dependent on the scale of the data. However, it is not symmetric like the MAE.

6.3.2 Test Dataset Comparison

The final model we had from Chapter 5 had the variables `newdata`. From this model we can use the `predict` function with the `newdata = test_dataset` option to use score the test dataset.

```
ames_lm <- lm(Sale_Price ~ . , data = train_reg)
test$pred_lm <- predict(ames_lm, newdata = test)

head(test$pred_lm)
```

```
##           1           2           3           4           5           6
## 166194.9 178562.0 233583.0 111924.2 215227.1 129452.5
```

To get predictions from the regularized regression models, a λ value must be selected. For the previous LASSO regression we will choose the largest λ value within one standard error of the minimum λ value to help reduce the number of variables. Again, we will use the `predict` function. The `s =` option is where we input the λ value. The `newx =` option is where we specify the `test` dataset.

```
test_reg$pred_lasso <- predict(ames_lasso, s = ames_lasso_cv$lambda.1se, newx = test)

head(test_reg$pred_lasso)
```

```
##           s1
## 1 156677.8
## 2 172432.5
## 3 239922.1
## 4 105713.6
## 5 200908.8
## 6 124913.5
```

Now we need to calculate the MAE and MAPE for each model for comparison.

```
test %>%
  mutate(lm_APE = 100*abs((Sale_Price - pred_lm)/Sale_Price)) %>%
  dplyr::summarise(MAPE_lm = mean(lm_APE))

## # A tibble: 1 × 1
##   MAPE_lm
##   <dbl>
## 1     12.3

test_reg %>%
  mutate(lasso_APE = 100*abs((Sale_Price - pred_lasso)/Sale_Price)) %>%
  dplyr::summarise(MAPE_lasso = mean(lasso_APE))

## # A tibble: 1 × 1
##   MAPE_lasso
##   <dbl>
## 1     13.4
```

From the above results, we see that the linear regression has a lower MAPE.

Once we have scored models with the test dataset, we should **not** go back to try and rebuild any models. We will use the model with the lowest MAE or MAPE. This number is also the number that we report on how well our model performs. No metrics on the training dataset should be reported for the performance of the model.

6.3.3 Python Code

CV in Regularized Regression

```
from sklearn.linear_model import LassoCV

ames_lasso2 = LassoCV(cv = 10, random_state = 0).fit(X, y)

ames_lasso2.alpha_

## 256031.19608623214

import time
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LassoLarsIC
from sklearn.pipeline import make_pipeline

start_time = time.time()
model = make_pipeline(StandardScaler(), LassoCV(cv = 10)).fit(X, y)
fit_time = time.time() - start_time
```

```

lasso = model[-1]
plt.semilogx(lasso.alphas_, lasso.mse_path_, linestyle=":")
plt.plot(
    lasso.alphas_,
    lasso.mse_path_.mean(axis=-1),
    color="black",
    label="Average across the folds",
    linewidth=2,
)
plt.axvline(lasso.alpha_, linestyle="--", color="black", label="alpha: CV estimator")

plt.xlabel(r"$\alpha$")
plt.ylabel("Mean square error")
plt.legend()
_ = plt.title(
    f"Mean square error on each fold: coordinate descent (train time: {fit_time:.1f} s)"
)

plt.show()

```

